

Laborator 7

Fire de execuție (thread-uri)

7.1 Scopul lucrării

Laboratorul curent prezintă conceptul de fir de execuție (eng. *thread*) în Linux. Vor fi prezentate apelurile de sistem pentru manipularea thread-urilor, precum și considerente practice în lucrul cu acestea.

7.2 Fundamente teoretice

În cadrul unui proces putem avea mai multe fire de execuție care rulează în paralel. Sistemul de operare ne permite să pornim un nou fir de execuție sau să așteptăm terminarea unuia.

7.2.1 Crearea unui thread și așteptarea terminării

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine) (void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
```

Atunci când utilizăm funcții legate de thread-uri, pe lângă includerea **pthread.h**, trebuie să includem și biblioteca **pthread**, în etapa de *link*-are a programului. Pentru aceasta, la apelul compilatorului (**gcc**) vom adăuga la linia de comandă opțiunea **-pthread**. Această opțiune marchează în executabilul produs faptul că se folosește cod din această bibliotecă și cere sistemului de operare să o încarce la execuția programului.

```
_____ Compilarea programelor cu pthread _____
gcc -Wall program.c -o program -pthread
```

7.2.1.1 pthread_create

Funcția **pthread_create()** ne permite să pornim un nou fir de execuție, care va executa codul funcției **start_routine**, transmisă prin al treilea argument.

Funcția executată de thread trebuie să aibă semnătura specificată, adică să primească un parametru de tip **void***, iar rezultatul returnat să fie **void***.

Primul argument al funcției **pthread_create()** este un pointer de tipul **pthread_t***, de obicei adresa unei variabile de tipul respectiv, în care funcția va completa id-ul thread-ului nou creat, în caz de succes.

Al doilea argument este un pointer la o structură de tip **pthread_attr_t**, în care putem specifica atributele thread-ului. În cazul în care dorim să folosim atributele implicite, acest pointer poate fi **NULL**.

Argumentul al treilea este un pointer la funcția ce va fi executată de thread, așa cum s-a explicat anterior. Argumentul al 4-lea este argumentul transmis funcției de thread. Pentru a putea transmite orice date funcției, argumentul este un pointer generic (`void*`), putând să fie adresa oricărei structuri definite de programator.

În caz de succes, funcția `pthread_create()` returnează 0.

7.2.1.2 pthread_join

Funcția `pthread_join()` ne permite să așteptăm terminarea execuției unui thread. Primul argument va fi id-ul thread-ului, obținut prin apelul funcției `pthread_create()`. Al doilea argument va fi un pointer de tip `void**`, care va fi modificat pentru a indica spre rezultatul funcției thread-ului. Acest argument e un pointer la un pointer de tip `void*`, deoarece funcția thread-ului returnează un rezultat de tip `void*`, de obicei adresa unei variabile (sau structuri) alocate dinamic, în care sunt stocate rezultatele funcției.

7.2.1.3 Exemplu

Vom începe cu un exemplu de program în care se creează un thread și se așteaptă terminarea lui.

```

1 | void *threadFn(void *unused)
2 | {
3 |     printf("Secondary thread\n");
4 |     return NULL;
5 | }
6 |
7 | int main()
8 | {
9 |     pthread_t tid = -1;
10 |
11 |     if(pthread_create(&tid, NULL, threadFn, NULL) != 0){
12 |         perror("Error creating thread");
13 |         return 1;
14 |     }
15 |     printf("Main thread\n");
16 |     pthread_join(tid, NULL);
17 |     return 0;
18 | }
```

Programul începe prin a defini funcția thread-ului `threadFn`. Așa cum s-a explicat mai sus, o funcție care poate fi folosită ca funcție de thread trebuie să primească un argument de tipul `void*` și să returneze `void*`. În acest exemplu argumentul nu se folosește și se returnează tot timpul `NULL`.

În funcția principală, se apelează `pthread_create` pentru a porni thread-ul. Primul argument este un pointer la variabila `tid`, declarată la linia 9, în care se va stoca identificatorul thread-ului. Acest identificator poate fi folosit ulterior pentru operații pe thread, cum ar fi așteptarea terminării acestuia (la linia 16). Al treilea argument este funcția de thread, care va fi executată pe un fir de execuție separat, după cum se poate observa în Figura 7.1.

Firul principal de execuție merge de la începutul până la sfârșitul programului. Firul secundar începe de la crearea thread-ului și continuă până la terminarea execuției funcției thread-ului. Terminarea execuției nu trebuie neapărat să coincidă cu apelul funcției `pthread_join` pe thread-ul principal. În Figura 7.2 se pot observa două scenarii:

- În Figura 7.2(a), funcția thread-ului își încheie execuția înainte să se apeleze `pthread_join`. În acest caz, la join rezultatul va fi disponibil imediat și thread-ul principal poate continua.
- În Figura 7.2(b), funcția thread-ului își încheie execuția după ce se apelează `pthread_join`. În acest caz, thread-ul principal este blocat, întrerupându-și execuția până la încheierea funcției thread-ului.

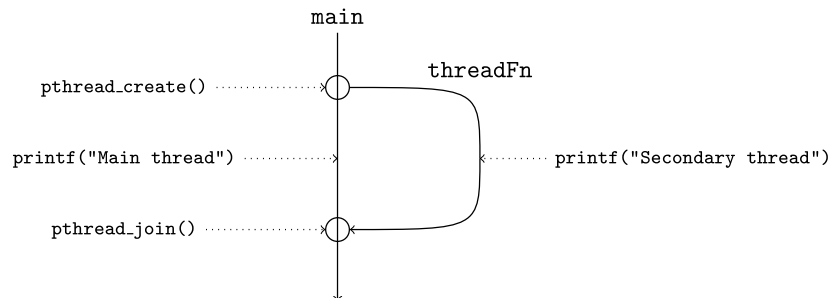
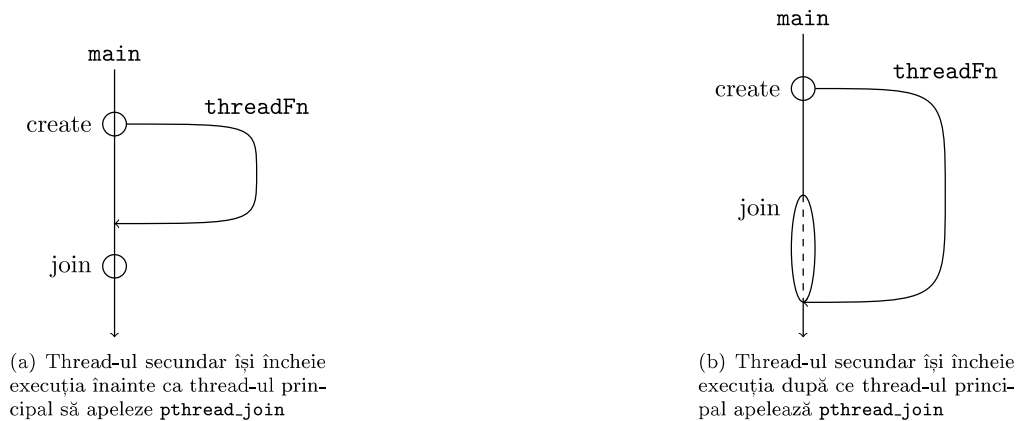


Figura 7.1: Ilustrarea execuției unui thread

Figura 7.2: Terminarea unui thread înainte sau după ce se apelează `pthread_join`

7.2.2 Transmiterea argumentelor unui thread

O funcție de thread va avea tot timpul prototipul `void *threadFn(void *)`, adică va primi întotdeauna un argument de tipul `void*`, iar rezultatul va fi tot de tipul `void*`. Acest tip oferă suficientă flexibilitate, deoarece un pointer de tip `void*` se poate converti în orice alt tip de pointer, inclusiv către o structură cu orice câmpuri am avea nevoie.

7.2.2.1 Exemplu

Dorim să scriem un program care generează un vector aleator, cu numere întregi și determină valoarea maximă din acesta folosind thread-uri.

Ideea de bază e să împărțim vectorul în mai multe bucăți de dimensiuni (aproximativ) egale, apoi fiecare thread să calculeze maximul dintr-o anumită bucată. Thread-ul principal va aștepta terminarea tuturor thread-urilor și va afișa valoarea maximă dintre maximele calculate de thread-uri.

7.2.2.2 Funcția de thread și parametri

```

1 | typedef struct {
2 |     int *arr;
3 |     int from;
4 |     int to;
5 | } TH_STRUCT;
6 |
7 | void *thread_fn_max(void *param)
  
```

```

8  {
9      TH_STRUCT *s = (TH_STRUCT*)param;
10     int i, max;
11     max = s->arr[s->from];
12     for(i=s->from; i<=s->to; i++){
13         if(s->arr[i] > max){
14             max = s->arr[i];
15         }
16     }
17     return (void*)(long)max;
18 }

```

Funcția unui thread trebuie să primească cel puțin următoarele informații:

- **arr** - pointer spre vectorul pe care se lucrează;
- **from** - indicele de început în vector pentru bucata alocată thread-ului;
- **to** - indicele de final în vector pentru bucata alocată thread-ului.

Cum astfel de funcții primesc un singur parametru de tipul **void***, vom defini o structură care conține informațiile dorite (liniile 1–5), iar funcția va primi ca parametru un pointer spre o astfel de structură.

În prima linie din funcție (linia 9) se face conversia de tip (eng. cast) a parametrului la pointer spre o structură **TH_STRUCT**, astfel încât să poată fi dereferențiat și să se poată extrage informațiile utile. În continuare, se parcurge vectorul între indicii **s->from** și **s->to**, calculându-se valoarea maximă în variabila **max**, de tip **int** (liniile 11–16). Această variabilă trebuie returnată de către funcție.

Funcția thread-ului returnează tipul **void***, care pe 64 bit ocupă 8 octeți, în timp ce pe 32 bit ocupă 4 octeți. În ambele cazuri, un întreg (care ocupă 32 bit pe majoritatea platformelor) încapă în capacitatea de reprezentare, trebuie doar să îi facem cast. Cast-ul de la linia 17 este unul dublu: **int**-ul se transformă întâi într-un **long**, apoi în **void***. Dacă am fi transformat direct din **int** în **void***, compilatorul ar fi afișat un warning de genul "**cast to pointer from integer of different size**", deoarece pe 64 bit, un întreg și un pointer au dimensiuni diferite.

7.2.2.3 Generarea vectorului aleator

```

1  #define SIZE_ARRAY 1000
2
3  void generate_random_array(int *v, int size)
4  {
5      int i;
6      for(i=0; i<size; i++){
7          v[i] = rand() % 1000000;
8      }
9  }
10
11 int main()
12 {
13     int v[SIZE_ARRAY];
14     ...
15
16     srand(time(NULL));
17     generate_random_array(v, SIZE_ARRAY);
18     ...
19 }

```

În funcția **main** se apelează **srand(time(NULL))** pentru inițializarea generatorului de numere aleatoare (linia 16), apoi se apelează funcția **generate_random_array** care completează valori

aleatoare în elementele vectorului.

7.2.2.4 Împărțirea vectorului între thread-uri

```

1 | #define SIZE_ARRAY 1000
2 | #define NR_THREADS 6
3 |
4 | int main()
5 | {
6 |     ...
7 |     TH_STRUCT params[NR_THREADS];
8 |     int i;
9 |     ...
10 |
11 |     for(i=0; i<NR_THREADS; i++){
12 |         params[i].arr = v;
13 |         if(i == 0){
14 |             params[i].from = 0;
15 |         }else{
16 |             params[i].from = params[i-1].to + 1;
17 |         }
18 |         params[i].to = params[i].from + SIZE_ARRAY / NR_THREADS - 1;
19 |         if(i < SIZE_ARRAY % NR_THREADS){
20 |             params[i].to++;
21 |         }
22 |
23 |         ... //create thread
24 |     }
25 |     ...
26 | }
```

Dorim să împărțim munca între thread-uri într-un mod cât mai echitabil. Dacă vectorul are `SIZE_ARRAY` elemente și avem `NR_THREADS` thread-uri, numărul de elemente procesate de fiecare thread ar trebui să fie `SIZE_ARRAY / NR_THREADS`. În cazul în care împărțirea nu se face exact (în exemplul curent, cu 1000 de elemente și 6 thread-uri, rămân 4 elemente neîmpărțite). Cel mai echitabil ar fi ca `SIZE_ARRAY % NR_THREADS` thread-uri să proceseze câte un element în plus.

În funcția `main` se declară vectorul `params`, ce va conține structurile cu parametri pentru fiecare thread. În bucla de la liniile 11–24 se setează valorile acestor parametri. Câmpul `arr` din fiecare structură va pointera spre vectorul `v` (linia 12). Primul thread va procesa elementele din vector începând cu indicele 0 (linia 14), iar celelalte thread-uri vor continua de la ultimul indice procesat de thread-ul anterior (linia 16). Capătul din dreapta al intervalului de indici va fi calculat adunând `SIZE_ARRAY / NR_THREADS` capătului din stânga (linia 18). În cazul în care suntem la primele `SIZE_ARRAY % NR_THREADS` thread-uri, adăugăm un element în plus la interval (liniile 19–21).

7.2.2.5 Lansarea și așteptarea thread-urilor

```

1 | int main()
2 | {
3 |     pthread_t tid[NR_THREADS];
4 |     TH_STRUCT params[NR_THREADS];
5 |     int i;
6 |     void *result;
7 |     int thMax, max = 0;
8 |     ...
9 |
10 |     for(i=0; i<NR_THREADS; i++){
```

```

11     ... // fill params[i]
12     pthread_create(&tid[i], NULL, thread_fn_max, &params[i]);
13 }
14 for(i=0; i<NR_THREADS; i++){
15     pthread_join(tid[i], &result);
16     thMax = (int)(long)result;
17     if(thMax > max){
18         max = thMax;
19     }
20 }
21 printf("The maximum value is %d.\n", max);
22
23 return 0;
24 }

```

Pentru a gestiona thread-urile cu care lucrăm, vom păstra identificatorul fiecăruia în vectorul `tid` declarat la linia 3. Structurile cu parametrii fiecărui thread au fost deja completate în secțiunea anterioară. Putem să creăm thread-urile în aceeași buclă în care se completează parametrii (liniile 10–13).

Funcția `pthread_create` primește ca prim argument un pointer spre `tid[i]` unde se va stoca id-ul thread-ului, `NULL` pentru atributele implicite, numele funcției de thread și un pointer la structura cu parametri `params[i]`.

Atunci când transmitem informații diferite thread-urilor, este foarte important ca fiecare thread să aibă parametrii într-o structură diferită, deoarece thread-urile primesc doar o referință la structură, nu o copie, și nu ar fi în regulă să transmitem adresa unei structuri pe care o vom modifica.

După ce s-au creat thread-urile trebuie să se aștepte după ele, folosind `pthread_join` (linia 15). Rezultatul thread-ului este de tipul `void*`, așa că trebuie să folosim un pointer la o variabilă de acest tip. Ulterior, pointer-ul respectiv poate fi transformat într-un întreg (linia 16) și comparat cu maximul existent.

Este foarte important ca așteptarea după thread-uri să se facă într-o buclă separată față de cea în care se creează. În Figura 7.3(a) se ilustrează execuția thread-urilor din codul de mai sus, în paralel. În prima buclă se creează toate thread-urile, apoi în a doua buclă se așteaptă terminarea execuției tuturor. O variantă greșită ar fi ca imediat după ce se creează un thread, să apelăm `pthread_join` pentru el, în aceeași buclă `for`. În acest caz, comportamentul ar fi cel ilustrat în Figura 7.3(b), obținându-se o performanță similară cu apelul direct al funcțiilor, fără să se profite de paralelism.

7.2.3 Încheierea execuției unui thread

```

#include <pthread.h>

void pthread_exit(void *retval);
int pthread_cancel(pthread_t thread);

int pthread_setcancelstate(int state, int *oldstate);
int pthread_setcanceltype(int type, int *oldtype);

```

7.2.3.1 pthread_exit

Un thread își încheie execuția atunci când funcția acestuia se termină, sau prin apelul `pthread_exit`, care primește ca parametru valoarea returnată de thread.

7.2.3.2 pthread_cancel

Un thread poate fi anulat (închis) și de către alt thread din același proces, apelând `pthread_cancel`, cu identificatorul thread-ului ca parametru.

În general, nu se recomandă această închidere “forțată” a unui thread și se preferă, pe cât posibil ca thread-urile să își încheie singure execuția. Atunci când un thread este închis din exterior, acesta

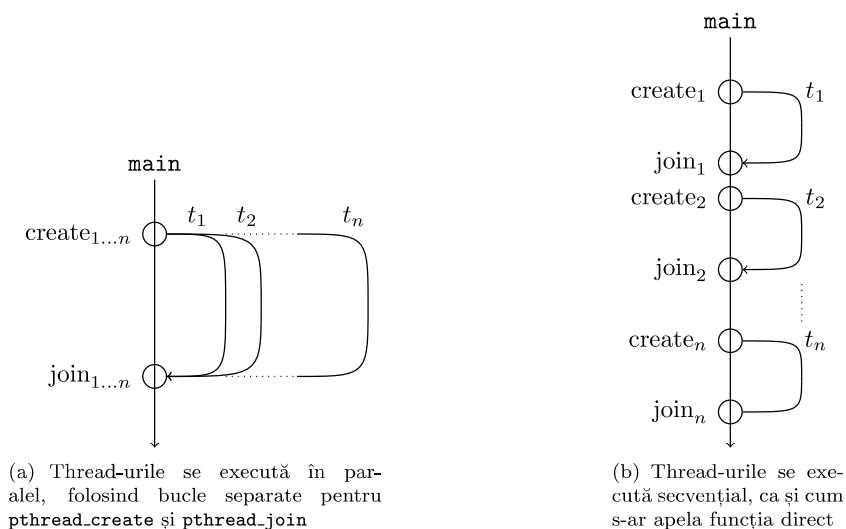


Figura 7.3: Importanța utilizării buclelor separate pentru `pthread_create` și `pthread_join`

poate lăsa resurse folosite într-o stare inconsistentă (de exemplu un fișier deschis și neînchis, sau un vector cu valori completat doar parțial). Totuși există și situații când un thread trebuie să poată fi închis de către alt thread.

Atunci când se apelează `pthread_cancel`, thread-ul țintă nu se va încheia instant, ci doar în momentul în care acesta atinge un punct de terminare (eng. *cancellation point*), în anumite apeluri de sistem, cum ar fi `open`, `close`, `read`, `write`, `sleep` (pentru lista completă puteți consulta pagina de manual).

7.2.3.3 `pthread_setcancelstate` și `pthread_setcanceltype`

În mod implicit, un thread poate fi anulat de alte thread-uri din același proces. Totuși, el își poate modifica starea, apelând `pthread_setcancelstate`. Parametrul `state` poate fi una din valorile:

- `PTHREAD_CANCEL_ENABLE` - thread-ul curent devine anulabil (implicit);
- `PTHREAD_CANCEL_DISABLE` - thread-ul curent nu se va putea anula prin apelul `pthread_cancel`.

Tot în mod implicit, dacă un thread este anulabil, închiderea acestuia se va face doar când se ajunge la un *cancellation point*. Se poate modifica acest tip de anulabilitate, apelând funcția `pthread_setcanceltype`. Parametrul `type` poate lua una din valorile:

- `PTHREAD_CANCEL_DEFERRED` - thread-ul curent se poate închide doar când atinge un punct de anulare (implicit);
- `PTHREAD_CANCEL_ASYNCYNCHRONOUS` - thread-ul curent se poate închide imediat ce se apelează `pthread_cancel`.

7.3 Mersul lucrării

7.3.1 Întrebări recapitulative

1. Cum se compilează în linia de comandă programele care folosesc thread-uri?
2. Ce primește ca argument și ce returnează o funcție de thread?
3. Cum procedăm atunci când funcția de thread trebuie să primească mai multe argumente?

4. Ce este greșit din punct de vedere al logicii programului în codul de mai jos?

```

1 | pthread_t tid;
2 | int i;
3 | for(i=0; i<NR_THREADS; i++) {
4 |     pthread_create(&tid, NULL, thread_fn, NULL);
5 |     pthread_join(tid, NULL);
6 | }
```

5. Când își încheie execuția un thread pentru care s-a apelat `pthread_cancel`?

7.3.2 Probleme rezolvate

1. (*l07p1.thread.c*) Programul este descris în secțiunea 7.2.1.3.
2. (*l07p2.max.c*) Programul este descris în secțiunea 7.2.2.1.

7.3.3 Probleme propuse

1. (35p) Scrieți un program care numără câte cifre de ‘1’ se găsesc în numerele de la 1 la n , folosind mai multe thread-uri. Pentru punctaj maxim, “munca” trebuie împărțită în mod echitabil între thread-uri (țineți cont că numerele mici au mai puține cifre ca numerele mari).
2. (10p) Pentru problema anterioară (dacă ați reușit să echilibrați volumul de calcule între thread-uri), identificați numărul optim de thread-uri pentru a minimiza timpul de execuție (pentru o valoare mare a lui n). Ce relație există între acel număr și numărul de nuclee pe care le are procesorul vostru? Pentru această problemă se va completa un fișier text care conține timpii de rulare pentru diverse valori ale numărului de thread-uri, pentru același n .
3. (25p) Scrieți un program care verifică care este numărul maxim de thread-uri care pot fi create simultan de un program (până ce `pthread_create` eșuează). Pentru ca numărătoarea să fie validă, thread-urile create nu trebuie să se închidă.
4. (30p) Scrieți un program care creează 9 thread-uri, fiecare din acestea executând o buclă infinită în care așteaptă un timp aleator între 1 și 5 secunde, apoi își afișează pe ecran numărul (de la 1 la 9). Pe thread-ul principal se citesc cifre de la tastatură. Atunci când se introduce cifra corespunzătoare unui thread, acesta va fi anulat. După ce s-au închis toate cele 9 thread-uri (trebuie să se și aștepte terminarea lor), se va închide și thread-ul principal.